

Contributors Guide

Casal2 Development Team



Casal2 Steering Group Technical Report
2026

Contributors Guide for use with Casal2 (v26.06 (2026-06-25))
<https://github.com/alistairdunn1/CASAL2.git:master>

Citation: Casal2 Development Team (2026). Contributors guide, v26.06 (2026-06-25). Casal2 Steering Group. *Technical Report*. 34 p. (Using source code from <https://github.com/alistairdunn1/CASAL2.git>:master)

Contents

1	Introduction	1
2	Creating a local repository	2
2.1	Git username and profile	2
2.2	Using git	2
2.3	Cloning a repository to your local machine	2
2.4	Create a GitHub fork to start making changes	3
2.5	Adding an SSH authentication key to your account on GitHub.com	3
3	Compiling Casal2	4
3.1	Overview	4
3.2	Building on Microsoft Windows	5
3.2.1	Prerequisite software	5
3.2.2	Pre-build requirements	7
3.2.3	Building Casal2	7
3.3	Building on Linux	8
3.3.1	Prerequisite software	8
3.3.2	Building Casal2	9
3.4	Troubleshooting	9
3.4.1	Third-party C++ libraries	9
3.4.2	Main code base	10
4	Casal2 build guidelines and validation	11
4.1	Casal2 coding practice and style	11
4.2	Units tests and model validation	11
4.2.1	Mocking specific classes	12
4.2.2	Internal mocking of simple models	12
4.2.3	External testing using test models	12
4.3	Verification	12
4.4	Reporting (optional)	13
4.5	Update the Casal2 User Manual(s)	13
4.6	Builds to pass before merging changes	13
5	Adding functionality into the Casal2 C++ code	15
6	Adding reports	23
7	Merging code changes from a branch to the master	24
8	Moderating code	26
9	Acknowledgements	27

1 Introduction

The Contributors Guide provides an overview for those who wish to compile the latest version of Casal2 and contribute to the Casal2 source code. Casal2 is open source and the Development Team encourages users to submit and contribute changes, bug fixes, and enhancements. This document is intended to provide a guide for those who wish to undertake these tasks.

The Casal2 source code is hosted on GitHub, and can be found at <https://github.com/Casal2/CASAL2>.

A release bundle which includes a binary (both windows and Linux operating systems), manual, examples, **R** package and other help guides can be downloaded at <https://github.com/Casal2/CASAL2/releases>. Note that the release versions on the repository can lag behind the latest source code as release versions are only compiled periodically.

This document provides a guide to understanding how to access the source code, how to compile and build, and the guidelines for submitting code changes to the main Casal2 repository. It also provides a simple introduction to setting up a GitHub profile and using git.

As a suggestion, it may be helpful to contact the Development Team before making any changes to the code. The Development Team can assist with ideas on how best to add functionality. Either contact them directly at `Casal2 Development Team` or open an issue at <https://github.com/Casal2/CASAL2/issues>.

If you have any questions or require more information, please contact the Casal2 Development Team at `Casal2 Development Team`.

2 Creating a local repository

Casal2 is stored on GitHub — and uses it to manage and store copies of the source code and ancillary files. This section is intended to be an introduction for new users to GitHub and git, and covers the following:

1. Registering a GitHub username
2. Using git
3. Cloning a repository to your local machine
4. Fork the master repository

2.1 Git username and profile

The first step is to create a username and profile on GitHub (if you do not already have one). Go to <https://www.github.com>, register a username, and set up a profile if you do not already have one. See the GitHub help for more information.

Once you have set up a GitHub account, you need to download git software (see below) so you can access and modify the code on your local machine.

2.2 Using git

You will need to install and use git software in order to download ('clone') the repository to your local machine and upload ('push') changes to your local repository. There are several git clients available and we suggest GitHub Desktop on Microsoft Windows as a good starting point.

Casal2 also requires a command line version of git in order to compile (see Section 3 for a complete description of the requirements of the Casal2 build environment).

Download a git client from git software and you ensure that the binary is included into your system path. This can be checked by opening a terminal (e.g., Powershell or command prompt on Microsoft Windows machines and typing in `git -v`.

2.3 Cloning a repository to your local machine

The Casal2 code, documentation, and other ancillary files is kept in the Master branch of the repository. Only the Casal2 Development Team have permission to add, delete, or change code directly. You can clone this repository to your local machine, but note that you won't be able to 'push' any of your changes back to the Casal2 Master repository. See the below in Section 2.4 for creating your own copy ('fork') of Casal2 on GitHub.

To clone the repository, navigate to <https://github.com/Casal2/CASAL2> and use the green code button, which is shown below 'clone'. There are multiple protocols (https, ssh etc.) for cloning highlighted in the red box. If you have GitHub Desktop installed, then choose this option and this will open the link in GitHub Desktop on your local machine (see the circled text in Figure 2.1).

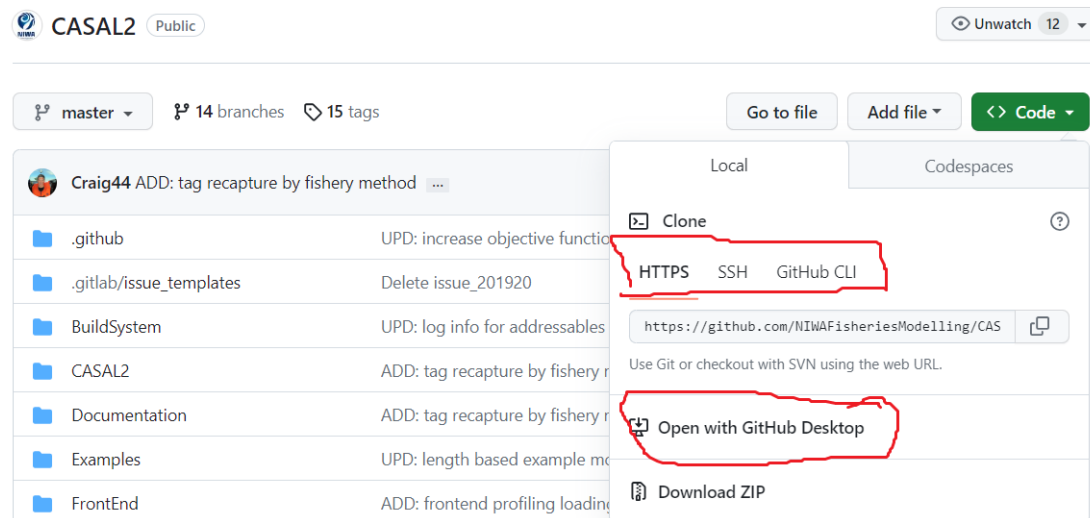


Figure 2.1: Cloning a repository

2.4 Create a GitHub fork to start making changes

This section covers how to fork a copy of the Master codebase of Casal2 into your GitHub account. If you wish to contribute to Casal2, you will need to fork a copy of Casal2 to your GitHub account. This provides a way for you to have a copy of the code and upload changes for your local version. Once you have finalised a change, it can then be submitted to the Casal2 Development Team for evaluation and potential inclusion into the main code base.

To do this, simply navigate to the Casal2 GitHub repository, and select fork, and then create a new fork.

As described above in Section 2.3, this fork can be cloned to your local machine and used to modify, add, or delete code. Note that any changes made here only effect your fork and will not modify the Master codebase at <https://github.com/Casal2/CASAL2>.

Once you have made a push, you will then be able to see the changes made. If you have any difficulty, check the GitHub documentation for help.

2.5 Adding an SSH authentication key to your account on GitHub.com

One of the reasons contributors may have difficulty pushing and pulling changes to Casal2 is because they don't have a SSH authentication key linked to their GitHub account. The error message may look like that shown in Figure 2.2. Please see the information from this link for detailed instructions on how to create a public SSH key on your computer and then link it to your GitHub account.

```
Initialized empty Git repository in `~/Users/username/Documents/cakebook/.git/`
Permission denied (publickey).
fatal: The remote end hung up unexpectedly
```

Figure 2.2: Example of the error message when SSH authentication has not been enabled

3 Compiling Casal2

This section describes how to set up the environment on your local machine that will allow you to build and compile Casal2. The build environment can be on either Microsoft Windows or Linux systems. At present the Casal2 build system supports Microsoft Windows 10+ and recent Linux systems. Apple OSX or other platforms are not currently supported.

3.1 Overview

The build system is made up of a collection of Python3 scripts that do the various tasks. These are located in `CASAL2/BuildSystem/buildtools/classes/`. Each Python script has its own set of functionality and undertakes one set of actions.

The top level of the build system can be found at `CASAL2/BuildSystem/`. In this directory you can run `doBuild.bat help` from a command window in Microsoft Windows systems or `./doBuild.sh help` from a terminal in Linux systems.

The script will take one or two parameters depending on what style of build you want to undertake. These commands allow the building of various stand-alone binaries, shared libraries, and the documentation. Note that you will need additional software installed on your system in order to build Casal2. The software requirements are described below.

A summary of all of the `doBuild` arguments can be found using the command `doBuild help` in the `BuildSystem` directory.

Note that the files `Casal2_build.bat` for Windows and `Casal2_build.sh` for Linux in the root folder contain all the calls in the correct order of `doBuild` required to successfully build Casal2, the documentation, the Windows installer (Windows) or the Debian installer (Linux), the R-Libraries, and run all the test cases and unit tests.

The arguments to `doBuild` are:

Usage: `doBuild <build_target> <argument>`

Clean, check, and help

<code>help</code>	Print out the <code>doBuild</code> help (this output)
<code>check</code>	Do a check of the build system
<code>clean</code>	Remove debug/release build files
<code>clean_all</code>	Remove all build files and ALL prebuilt binaries
<code>version</code>	Build the current version files for C++, R, and L ^A T _E X

Build required libraries (DLLs/shared objects for Casal2)

`thirdparty` Build the third party libraries

`<option>` Optionally specify the target third party library to build, either `adolc` or `betadiff` (default is none)

Build development and test versions (for development builds only)

`release` Build stand-alone release executable

`<option>` Optionally specify the target third party library to build, either `adolc` or `betadiff` (default is none)

`debug` Build stand-alone debug executable

<option> Optionally specify the target third party library to build, either adolc or betadiff (default is none)

test Build stand-alone unit tests executable

unittests Run the unit tests (requires that ‘test’ has been built)

modelrunner Run the test case models

Build the Casal2 end-user application

library Build shared library for use by front end application

<argument> Required argument to specify the target library to build: release, adolc, betadiff, or test

frontend Build Casal2 front end application

Create the archive, R library, documentation, and the Casal2installers

documentation Build the Casal2 user manuals

rlibrary Create the R library

archive Create a zipped archive of the Casal2 application.

<true> if specified build skips everything but frontend

installer Create the Microsoft Windows installer package

deb Create Linux Debian installer

Build output directories

The outputs from the build system commands will be placed in sub-folders of CASAL2/BuildSystem/bin/<operatingsystem>/<build_type>

For example:

CASAL2/BuildSystem/windows/debug

CASAL2/BuildSystem/windows/library_release

CASAL2/BuildSystem/windows/thirdparty/

CASAL2/BuildSystem/linux/library_release

3.2 Building on Microsoft Windows

3.2.1 Prerequisite software

The building of Casal2 requires additional build tools and software, including Python3, git version control, C++ compiler, L^AT_EX compiler, and a Windows package builder. Casal2 can require specific implementations of these packages and versions in order to build without modifying the build scripts.

C++ and Fortran compiler

An up-to-date C++ and Fortran compiler is required.

We suggest the WinLibs version from <https://www.winlibs.com>. This can be installed using curl (if installed) and python with the following command from a command prompt in Windows:

```
curl -Ls https://github.com/bwoodsend/setup-winlibs-action/raw/v1/install.py |  
python - --add-to-path
```

Casal2 is designed to compile under GCC on Microsoft Windows and Linux. While it may be possible to build the package using different compilers, the Casal2 Development Team does provide any assistance or recommendations. We recommend using the 64-bit version 14.2.0.

Note that a common issue that is often encountered is having a different C++ compiler in your path when attempting to compile. For example, rtools includes a version of the GCC compiler. We recommend removing these from your path prior to compiling.

GIT version control

Source: Command line GIT from <https://www.git-scm.com/downloads>.

Note that all of the Casal2 source files must be in a local GIT repository so the version and repository name can be evaluated. Casal2 automatically adds the version details to its files based on the GIT version of the latest commit to the source repository. This includes the name of source repository that was used. The command line version of GIT is used to generate the version details.

MiKTeX L^AT_EX processor

Source: Portable version from <http://www.miktex.org/portable>.

The main user documentation for Casal2 is a PDF document generated from L^AT_EX. The L^AT_EX syntax sections of the documentation are generated, in part, directly from the code. In order to generate the user documentation, you will need the MiKTeX L^AT_EX compiler.

A number of additional L^AT_EX styles are used — these will usually be identified during the doBuild process and can be installed as required.

7-Zip

Source: 7-Zip from <http://www.7-zip.org/download.html>.

The build system calls 7zip.exe to unzip files in the build system; it is advised to have this in the path.

Python3

Source: Python3 from <https://www.python.org/downloads/windows>

Python is used to run the build scripts and set the required environment variables required to build Casal2.

Python modules

There are a number of Python3 modules that are required to build Casal2. These can be installed with `python -m pip install module-name`. For example, You may need to install `datetime`, `re`, and `distutils` Python modules.

Inno setup installer build (optional)

Source: Inno Setup 5 from <http://www.jrsoftware.org/isdl.php>

If you wish to build a Microsoft Windows compatible Installer for Casal2 (recommended) then you will need the 'Inno Setup 5' application installed on the machine. The installation path must be `C:\ProgramFiles(x86)\InnoSetup5\` in order for the build scripts to find and use it.

3.2.2 Pre-build requirements

Prior to building Casal2 you will need to ensure you have both G++ and GIT in your path. You can check both of these by typing the following commands and checking that they return the correct version number:

```
g++ --version
```

```
git --version
```

This also allows you to check that there are no alternative versions of a GCC compiler that may confuse the Casal2 build. It's also worth checking to ensure GFortran has been installed with the G++ compiler by typing:

```
gfortran --version
```

If you wish to build the documentation, `bibtex` will also need to be in the path, e.g., to check, try:

```
bibtex --version
```

3.2.3 Building Casal2

The build process is relatively straightforward. Before you start the build process, you can run `doBuild check` from the command prompt to check if your build environment is complete. Make sure that you are within `CASAL2/BuildSystem/` to run `doBuild`.

`doBuild check` will summarise Windows environment `PATH` as a part of its output, and this can be used to check that the paths for `g++` and `gfortran` and the `g++` point to where the correct version of GCC is installed.

The build process is as follows:

1. Download a clone of the code on your local machine
2. Navigate to the `BuildSystem` folder in `CASAL2/BuildSystem`
3. You need to build the third party libraries with the following commands from the command prompt:
 - `doBuild thirdparty`
4. You need to build the binary you want to use:
 - `doBuild release`
5. You can build the documentation if you want:
 - `doBuild documentation`

3.3 Building on Linux

This guide has been written against a fresh install of Ubuntu 20.04. With Ubuntu we use apt-get to install new packages. You'll need to be familiar with the package manager for your distribution to correctly install the required prerequisite software. For this you will require administrator level access.

3.3.1 Prerequisite software

G++ compiler

If gfortran is not installed, install this with: `sudo apt-get install gfortran`.

GIT version control

Git may not be installed by default and it can be installed with `sudo apt-get install git`

Note that all of the Casal2 source files must be in a local GIT repository so the version and repository name can be evaluated. Casal2 automatically adds the version details to its files based on the GIT version of the latest commit to the source repository. This includes the name of source repository that was used. The command line version of GIT is used to generate the version details.

CMake

CMake is required to build multiple third-party libraries and the main code base. You can do this with `sudo apt-get install cmake`

Python

Python3 is used to run the build scripts and set the required environment variables required to build Casal2. This is usually installed by default on Linux systems, but if not, it can be installed using: `sudo apt-get install python3`

Python modules

There are a number of Python3 modules that are required to build Casal2. These can be installed with `sudo apt-get install module-name`. For example, You may need to install `datetime`, `re`, and `distutils` Python modules.

L^AT_EX

L^AT_EX on Linux is required, and the Texlive L^AT_EX processor is recommended. This can be installed with:

```
sudo apt-get install texlive-binaries
sudo apt-get install texlive-latex-base
sudo apt-get install texlive-latex-recommended
sudo apt-get install texlive-latex-extra
```

Alternatively you can install the complete package with `sudo apt-get install texlive-full`

A number of additional L^AT_EX styles are used — these will usually be identified doing the doBuild process and can be installed as required.

3.3.2 Building Casal2

The build process is relatively straightforward. You can run `./doBuild.sh` check to see if your build environment is ready.

1. Download a clone of the code on your local machine
2. Navigate to the BuildSystem folder in CASAL2/BuildSystem
3. You need to build the third party libraries with:
 - `./doBuild.sh thirdparty`
4. You need to build the binary you want to use:
 - `./doBuild.sh release`
5. You can build the documentation:
 - `./doBuild.sh documentation`

3.4 Troubleshooting

3.4.1 Third-party C++ libraries

It's possible that there will be build errors or issues building the C++ third-party libraries. If you encounter an error, then check the log files to locate the source of the problem. Each third-party build system stores a log of everything that was done. The files will be named

- `casal2_unzip.log`
- `casal2_configure.log`
- `casal2_make.log`
- `casal2_build.log`
- ...etc.,

Some of the third-party libraries require very specialised environments for compiling under GCC on Windows. These libraries are packaged with MSYS (MinGW Linux style shell system). The log files for these will be found in `ThirdParty/<libraryname>/msys/1.0/<libraryname>/`

e.g., `ThirdParty/adolc/msys/1.0/adolc/ADOL-C-2.5.2/casal2_make.log` and `ThirdParty/boost/boost_1_58_0/casal2_build.log`

A common issue when running `doBuild thirdparty` are Python error messages about missing modules, e.g., `ModuleNotFoundError: No module named 'dateutil'`. This type of error message indicates that a Python module (library) is missing and will need to be installed. For instance, to install the 'dateutil' module, type the following into a command prompt or terminal window: `pip3 install python-dateutil`.

3.4.2 Main code base

If the unmodified code base does not compile, the most likely cause is an issue with the third-party libraries not being built correctly. As updates and revisions are outside the control of the Development Team, problems can arise that may require the developers of the third party libraries to resolve first. However, versions of these libraries are included in the Casal2 source code and these should work. For any specific issues contact a local expert with regard to your specific system environment, or else the Casal2 Development Team for help.

4 Casal2 build guidelines and validation

4.1 Casal2 coding practice and style

Casal2 is written in C++ and uses the Google C++ style guide (see <https://google.github.io/styleguide/cppguide.html>).

In general when editing or writing code for Casal2

1. Using consistent indentations inside functions and loops, and descriptive and human readable variable or function names.
2. Use of the characters ‘_’ on the end of class variables defined in the .h files.
3. Annotate and comment the code, especially where it would help another contributor understand the program logic and rationale.
4. Add descriptive log messages to aid in debugging and checking of the program logic flow.
5. Implement unit tests, internal models, and external models to test and validate the new or changed functionality.
6. Document the functionality in the Casal2 User Manual(s).

Casal2 allows printing of logging messages at runtime using the `-loglevel` command line argument. The levels of logging in Casal2 are:

- `LOG_MEDIUM()` usually reserved for iterative functionality (e.g. estimates during estimation phase)
- `LOG_FINE()` the level of reporting between an actual report and a fine scale detail that end users are not interested in (Developers)
- `LOG_FINEST()` Minor fine scale details within a function or routine.
- `LOG_TRACE()` put at the beginning of every function

e.g., to run Casal2 with logging, use

```
casal2 -r -loglevel finest > my_run.log 2> my_run.err
```

This will output all the logged information to `my_run.err`.

4.2 Units tests and model validation

The Casal2 development places an emphasis on maintaining software integrity and reproducibility between revisions. Casal2 uses model validations and built in unit tests to validate and verify the code each time Casal2 is compiled and built.

There are three different validation approaches in Casal2. These are:

1. Mocking specific classes.
2. Implementing internal models (implemented in C++ source code with extension `.Test.cpp`) that have variable test cases for specific classes.

3. Implementing externally run models (found in the `TestModel` folder) that are validated to generate expected output.

To implement mocking of classes and internal models, Casal2 uses the Google testing framework and the Google mocking framework.

To implement testing of full models, input configuration files are run using the compiled Casal2 binaries, and the output compared with expected output using `@assert` commands.

4.2.1 Mocking specific classes

Classes are unit tested using unit tests that are a part of the source code. These are designed to check the components of the code to validate that functions provide expected output. These unit tests are run each time Casal2 is compiled.

When adding unit tests, they should to be developed and tested outside of Casal2. This gives confidence that the test does not contain any calculation errors.

Mocking specific classes is used to validate specific functionality and is encouraged because it is the easiest to isolate simple errors that may be introduced with code changes.

As examples, see (i) the file `VonBertalanffy.Test.cpp` mocks the `von Bertalanffy` age-length class and tests the mean length calculation, and (ii) the `Partition` class has the file `Partition.Test.cpp` that validates user inputs and model expectations.

4.2.2 Internal mocking of simple models

Mocking of simple models is done using a number of internal models. Most of the functionality for implementing these are in the source folder `/CASAL2/source/TestResources`.

These implements simple models and run test cases with differing class implementations by running an internal empty model and testing the output of classes from a model run.

As examples, see (i) the `LogNormal.Test.cpp` in the `Projects` class that test the lognormal distribution when used for projections, and (ii) the `TagByLength` process in `TagByLength.Test.cpp` that tests functionality of the tagging process.

4.2.3 External testing using test models

External tests are run following compilation using the Python `modelrunner.py` scripts (i.e., using the `DoBuild modelrunner` script in the `BuildSystem` folder). These models are used to test model runs, minimisation routines, and MCMC output.

The test model input configuration files are located in the `TestModel` folder and the command calls to run these are in the `modelrunner.py` script.

Contributors are encouraged to add additional models to the list of test models as these be used to validate the combined functionality of a range of interrelated commands and subcommands in Casal2.

4.3 Verification

After Casal2 executes `Validate` and `Build` it runs sanity checks in the `verify` state. These are business rules that can be checked across the entire system. This can be useful to suggest dependencies or configurations. For an example see the directory `Processes\Verification\` in the source code.

4.4 Reporting (optional)

Currently Casal2 has reports that are **R** compatible, i.e., all output reports produced by Casal2 can be read into **R** using the standard **CASAL2 R** package. If you create a new report or modify an old one, you must follow the standard so that the report is **R** compatible.

All reports must start with, `*label (type)` and end with, `*end`

Depending on what type of information you wish to report, will depend on the syntax you need to use. For example

{d} (Dataframe) Report a dataframe

```
*estimates (estimate_value)
values {d}
process[Recruitment_BOP].r0 process[Recruitment_ENLD].r0
2e+006 8e+006
*end
```

{m} (Matrix) Report a matrix

```
*covar (covariance_matrix)
Covariance_Matrix {m}
2.29729e+010 -742.276 -70160.5
-110126 -424507 -81300
-36283.4 955920 -52736.2
*end
```

{L} (List) Report a List

```
*weight_one (partition_mean_weight)
year: 1900
ENLD.EN.notag {L}
mean_weights {L}
0.0476604 0.111575 0.199705
end {L}
age_lengths {L}
12.0314 16.2808 20.0135
end {L}
end {L}
*end
```

4.5 Update the Casal2 User Manual(s)

Contributors will need to add or modify sections of the user manual(s) to document their changes. This includes the section that describes the methods and the section where the specific syntax is defined.

4.6 Builds to pass before merging changes

Once you have made changes, you must run the following before your changes can be included in the master code.

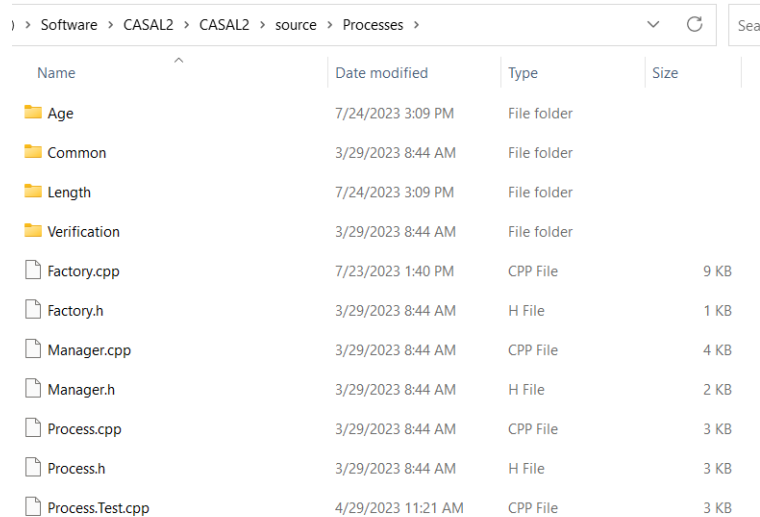
- Build the unittest version. See Section 3 for how to build unittest depending on your system.

- Run the standard and new unit tests to check that they all pass. To do this first compile the test executable using the script `DoBuild test`. Then move to the directory with the location of the executable (`BuildSystem/bin/OS/test`) and run it (open a command terminal and run `casal2`) to check all the unit-tests pass.
- Test that the debug and release of Casal2 compiles and runs with `DoBuild debug`
- Run the second phase of unit tests (requires that the debug version is built). This runs the tests that comprise of complete model runs using `DoBuild modelrunner`
- Build the archive using `DoBuild archive` which builds all required libraries. There are small nuances between Double classes, especially when reporting the class that mean seemingly simple changes can sometimes cause a break in the full build.

5 Adding functionality into the Casal2 C++ code

The following shows a sequence of figures and text of an example, where we demonstrate how to modify the Casal2 C++ code by adding a new child process called `SurvivalConstantRate`. Although we will be showing adding new process functionality for a process we will explain it in a generalised way so that this can be translated into adding a new selectivity, observation, likelihood, Projection, MCMC, minimiser, time varying class etc.

To add a new process you start by going into the `CASAL2\CASAL2\source\processes\`.



Name	Date modified	Type	Size
Age	7/24/2023 3:09 PM	File folder	
Common	3/29/2023 8:44 AM	File folder	
Length	7/24/2023 3:09 PM	File folder	
Verification	3/29/2023 8:44 AM	File folder	
Factory.cpp	7/23/2023 1:40 PM	CPP File	9 KB
Factory.h	3/29/2023 8:44 AM	H File	1 KB
Manager.cpp	3/29/2023 8:44 AM	CPP File	4 KB
Manager.h	3/29/2023 8:44 AM	H File	2 KB
Process.cpp	3/29/2023 8:44 AM	CPP File	3 KB
Process.h	3/29/2023 8:44 AM	H File	3 KB
Process.Test.cpp	4/29/2023 11:21 AM	CPP File	3 KB

Figure 5.1: Example of how the main C++ process classes are structured in Casal2

This folder shown in Figure 5.1 contains a general layout of all main classes in Casal2. It contains a folder named `Age`, `Length`, `Common`, `Verification`, `Factory.cpp`, `Factory.h`, `Manager.cpp`, `Manager.h`, `Process.cpp`, `Process.h`. The folder `Age` contains all the current age-based processes implemented, `Length` contains all the current length-based processes and `Common` contains all processes that can be used in both age or length models. The `Factory` source code creates all the processes during runtime. The `Manager` source code manages the class; it can build pointers to specific processes to share information across classes. The `Process` source code contains base functionality that is inherited by all child classes (inheritance is a major concept in C++ and it is advised have some knowledge of the concept). Before creating a new child look at the parent (in this example `Process.cpp`, `Process.h`), to see the functionality you don't have to add in your child because it is already done at the parent level. The following figure shows the constructor in `Process.cpp`.

From Figure 5.2 the process parent class does not do much — it assigns a label and type for each child that will be created and a report subcommand. The point of this is to look at the parent class to reduce duplicating functionality in the child class.

```

#include "Model/Managers.h"
#include "Model/Model.h"
#include "Reports/Manager.h"
#include "Reports/Children/Process.h"

// namespaces
namespace niwa {

/**
 * Default constructor
 */
Process::Process(Model* model) : model_(model) {
    parameters_.Bind<string>(PARAM_LABEL, &label_, "Label", "");
    parameters_.Bind<string>(PARAM_TYPE, &type_, "Type", "", "");
    parameters_.Bind<bool>(PARAM_PRINT_REPORT, &create_report_, "Generate parameter report", "", false);
}

/**
 * Call the validation method for the child object of this process and
 * set some generic variables.
 */
void Process::Validate() {
    parameters_.Populate();

    if (block_type_ != PARAM_PROCESS && block_type_ != PARAM_PROCESSES) {
        if (type_ != "")
            type_ = block_type_ + "_" + type_;
        else
            type_ = block_type_;

        block_type_ = PARAM_PROCESS;
    }

    if (process_type_ == ProcessType::kUnknown)
        LOG_CODE_ERROR() << "process_type_ == ProcessType::kUnknown for label: " << label();

    DoValidate();
}

```

Figure 5.2: Example of a C++ parent process class for SurvivalConstantRate

We will be returning to the factory source code later, but for now let's get adding the new process. Enter the Children folder and create C++ source code labelled `SurvivalConstantRate.cpp` and `SurvivalConstantRate.h` (See Figure 5.3). These files were should exist in the source code, but fell free to delete them and go through the process of adding the process to see how easy this process is. One way to start is by copying an existing process, renaming it, and using that as the basis for changes.

	RecruitmentBevertonHolt.Test.cpp	25/07/2016 7:50 a...	CPP File	9 KB
	RecruitmentConstant.cpp	1/03/2016 2:04 p.m.	CPP File	5 KB
	RecruitmentConstant.h	1/03/2016 2:04 p.m.	H File	2 KB
	RecruitmentConstant.Test.cpp	1/03/2016 2:04 p.m.	CPP File	3 KB
	SurvivalConstantRate.cpp	29/07/2016 11:23 ...	CPP File	6 KB
	SurvivalConstantRate.h	29/07/2016 11:23 ...	H File	2 KB
	TagByAge.cpp	1/03/2016 2:04 p.m.	CPP File	18 KB
	TagByAge.h	1/03/2016 2:04 p.m.	H File	3 KB
	TagByAge.Test.cpp	24/07/2016 10:00 ...	CPP File	14 KB
	TagByLength.cpp	15/07/2016 1:26 p...	CPP File	16 KB

Figure 5.3: Example showing the C++ child process classes for SurvivalConstantRate

Once you have done that for each child class you may have to write code for the following functions: `DoValidate()`, `DoBuild()`, `PreExecute()`, `DoExecute()`, `PostExecute()`, `DoReset()`. To understand what these all do Figure 5.4 shows the state transition of Casal2.

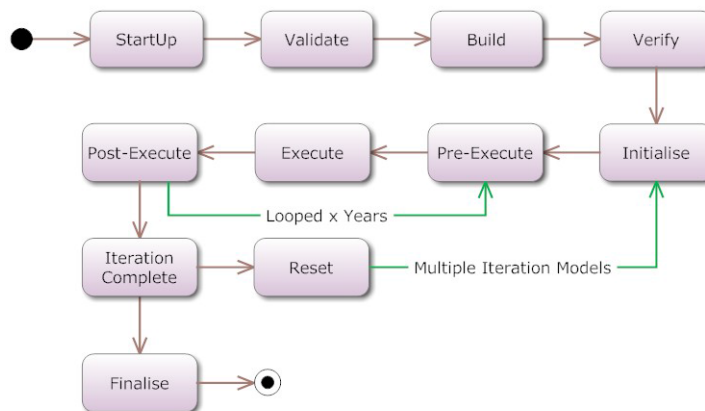


Figure 5.4: Diagram of the state transitions of Casal2.

Now we describe the purpose of each transition and this will give you an idea of what you should be incorporating into each function.

StartUp The model is in the blank start and the configuration system is loading the configuration files and parsing any extra inputs.

Tasks completed:

- Parse command line
- Parse configuration file
- Load plugins
- Load estimate values from input files

Validate All user configurations have been loaded at this point. Now the model will go through every object that has been created and check that the parameters given to them.

This step will ensure every object in the model has sufficient parameters to be executed without causing a system fault or error in the model.

This state will not check the values to ensure they are logical in relation to an actual model. They will only test that they exist and meet minimum requirements to execute a model.

At the end of the validate stage each object should be internally consistent. No lookups or external references are allowed to be formed during this stage.

Build The build phase is where the system will build relationships between objects that rely on each other. Because validation has been completed, each object in it's self-contained configuration is okay.

This phase generally assigns values to pointers for objects so they don't need to do lookups of objects during execution phases.

Verify At this point pre-defined configurations are checked against the model's current configuration to verify if the model makes sense logically. These are business rules being applied to the model to help ensure the model is "sensible" e.g. you have an ageing and a mortality process in your annual cycle.

PreExecute Pre-Execution happens at the beginning of a time step. This allows objects to calculate values based on the partition state before any of the other processes in the time step are executed.

Execute This is the general work method of the model and where all of the processes will be run against the partition.

PostExecute This is executed at the end of a time step after all of the processes and associated objects have been executed. This is typically used for things like reports and derived quantities.

IterationComplete This is executed at the end of every model run. This is only useful when the model is in a multiple-iteration mode (e.g MCMC or estimation). After every model iteration this state is triggered.

Reset If the model has to run multiple iterations then the reset state is used to reset everything back in to a state where the model can be re-executed without any legacy data remaining.

This state allows us to run multiple iterations of the model without having to re-process the configuration information or de-allocate/re-allocate large amounts of memory.

Finalise Finalise will happen after all iterations of the model have been completed.

Coming back to our example, if we look in `SurvivalConstantRate.h` we will see the following setup, as shown in Figure 5.5.

We can see that we are implementing `DoValidate()`, `DoBuild()`, `DoExecute()`, `DoReset()`, which are public functions and we define our variables as private. This idea of structuring classes by public, private and protected is known as encapsulation, and is another important C++ concept. Our variables will be stored in memory for the entire model run so you must make decisions on whether a variable needs to persist for the entire model run or can be temporarily created and destroyed at execution time. Note: the '_' at the end the variable names are used (by convention) to indicate a variable defined in the .h file.


```

// namespaces
namespace niwa {
class Selectivity; // Forward declare Selectivity class

namespace processes {
namespace accessor = niwa::partition::accessors;
using utilities::OrderedMap;
/**
 * Class Definition
 */
class SurvivalConstantRate : public niwa::Process {
public:
    // Methods
    SurvivalConstantRate(Model* model);
    virtual ~SurvivalConstantRate() = default;
    void DoValidate() override final;
    void DoBuild() override final;
    void DoReset() override final;
    void DoExecute() override final;

private:
    // Members
    vector<string> category_labels_;
    vector<Double> s_input_;
    OrderedMap<string, Double> s_;
    vector<Double> ratios_;
    map<unsigned, Double> time_step_ratios_;
    vector<vector<Double>> survival_rates_;
    vector<string> selectivity_names_;
    accessor::Categories partition_;
    vector<Selectivity*> selectivities_;
};

```

Figure 5.5: Example showing the definitions of the Casal2 standard functions in the class header file

Moving into `SurvivalConstantRate.cpp`, if we look at the constructor, we can see that this describes input parameters that are supplied by the user, and which variables are estimable. This is applied by the `RegisterAsEstimable()` function (See Figure 5.6).

```

SurvivalConstantRate::SurvivalConstantRate(Model* model)
: Process(model),
  partition_(model) {
    LOG_TRACE();
    process_type_ = ProcessType::kMortality;
    partition_structure_ = PartitionStructure::kAge;

    parameters_.Bind<string>(PARAM_CATEGORIES, &category_labels_, "List of categories", "");
    parameters_.Bind<Double>(PARAM_S, &s_input_, "Survival rates", "");
    parameters_.Bind<Double>(PARAM_TIME_STEP_RATIO, &ratios_, "Time step ratios for S", "", true);
    parameters_.Bind<string>(PARAM_SELECTIVITIES, &selectivity_names_, "Selectivity label", "");

    RegisterAsEstimable(PARAM_S, &s_);
}

```

Figure 5.6: Example showing the `RegisterAsEstimable()` function in the constructor

When you see keywords that begin with `PARAM`, for example `PARAM_CATEGORIES`, these will be defined in `CASAL2\CASAL2\source\English_UK.h`.

Moving on to the `DoValidate()` function in Figure 5.7. The purpose of this function is to validate the user's inputs and expand inputs to allow short hand syntax. You can see the annotation of what is happening in the function. This is considered one of the coding commandments **Thou shall annotate thy code**.

```

/**
 * Validate our Survival Constant Rate process
 *
 * - Validate the required parameters
 * - Assign the label from the parameters
 * - Assign and validate remaining parameters
 * - Duplicate 's' and 'selectivities' if only 1 vale specified
 * - Check s is between 0.0 and 1.0
 * - Check the categories are real
 */
void SurvivalConstantRate::DoValidate() {
    category_labels_ = model->categories()->ExpandLabels(category_labels_, parameters_.Get(PARAM_CATEGORIES));

    // If one S supplied expand for each category
    if (s_input_.size() == 1)
        s_input_.assign(category_labels_.size(), s_input_[0]);
    // Do the same for selectivity labels
    if (selectivity_names_.size() == 1)
        selectivity_names_.assign(category_labels_.size(), selectivity_names_[0]);
    //Check we have equal category labels as survival rates
    if (s_input_.size() != category_labels_.size()) {
        LOG_ERROR_P(PARAM_S)
            << ": Number of Ms provided is not the same as the number of categories provided. Expected: "
            << category_labels_.size() << " but got " << s_input_.size();
    }
    //Check we have equal category labels to selectivity labels
    if (selectivity_names_.size() != category_labels_.size()) {
        LOG_ERROR_P(PARAM_SELECTIVITIES)
            << ": Number of selectivities provided is not the same as the number of categories provided. Expected: "
            << category_labels_.size() << " but got " << selectivity_names_.size();
    }

    // Validate our S's are between 1.0 and 0.0
    for (Double s : s_input_) {
        if (s < 0.0 || s > 1.0)
            LOG_ERROR_P(PARAM_S) << ": s value " << AS_DOUBLE(s) << " must be between 0.0 and 1.0 (inclusive)";
    }
    // Assign survival rates to a map s_
    for (unsigned i = 0; i < s_input_.size(); ++i)
        s_[category_labels_[i]] = s_input_[i];

    // Check categories are real
    for (const string& label : category_labels_) {
        if (!model->categories()->IsValid(label))
            LOG_ERROR_P(PARAM_CATEGORIES) << ": category " << label << " does not exist. Have you defined it?";
    }
}

```

Figure 5.7: Example of the validations in the `DoValidate()` function

Another nice inclusion to the Casal2 source code is it's error checking and messages to the users. The following list is of all the allowable types of errors that Casal2 can perform.

- `LOG_WARNING()` << "warning message"; if triggered print the warning message at the end of model run to, warn the user of a change or implication in the model.
- `LOG_ERROR()` << "error message"; After validate and build stop execution and print error message.
- `LOG_ERROR_P(PARAM_KEYWORD)` << "error message"; After validate and build stop execution and print error message with location of the parameter `PARAM_KEYWORD` in the configuration files.
- `LOG_FATAL()` if triggered halt execution and print error message.
- `LOG_FATAL_P(PARAM_KEYWORD)` << "error message"; If triggered stop execution and print error message with location of the `PARAM_KEYWORD` in the configuration files.
- `LOG_CODE_ERROR()` << "error message"; if triggered quit Casal2 with a message like contact the development team this is a bug in the software.

We encourage you to put these all throughout the validate and build functions, to help users correctly specify models and catch silly input sequences.

Moving on to the `DoBuild()` function. The purpose of the build function is to create pointers and relationships with other classes that will be required during execution. For example, in this process we will require access to the partition, time step and selectivity classes.

```
/**
 * Build any runtime relationships
 * - Build the partition accessor
 * - Build our list of selectivities
 * - Build our ratios for the number of time steps
 */
void SurvivalConstantRate::DoBuild() {
    partition_.Init(category_labels_);

    for (string label : selectivity_names_) {
        Selectivity* selectivity = model_>managers().selectivity()->GetSelectivity(label);
        if (!selectivity)
            LOG_ERROR_P(PARAM_SELECTIVITIES) << ": selectivity " << label << " does not exist. Have you defined it?";

        selectivities_.push_back(selectivity);
    }

    /**
     * Organise our time step ratios. Each time step can
     * apply a different ratio of S so here we want to verify
     * we have enough and re-scale them to 1.0
     */
    vector<TimeStep*> time_steps = model_>managers().time_step()->ordered_time_steps();
    LOG_FINEST() << "time_steps.size(): " << time_steps.size();
    vector<unsigned> active_time_steps;
    for (unsigned i = 0; i < time_steps.size(); ++i) {
        if (time_steps[i]->HasProcess(label_))
            active_time_steps.push_back(i);
    }

    if (ratios_.size() == 0) {
        for (unsigned i : active_time_steps)
            time_step_ratios_[i] = 1.0;
    } else {
        if (ratios_.size() != active_time_steps.size())
            LOG_ERROR_P(PARAM_TIME_STEP_RATIO) << " length (" << ratios_.size()
                << ") does not match the number of time steps this process has been assigned to (" << active_time_steps.size() << ")";

        for (Double value : ratios_) {
            if (value <= 0.0 || value > 1.0)
                LOG_ERROR_P(PARAM_TIME_STEP_RATIO) << " value (" << value << ") must be between 0.0 (exclusive) and 1.0 (inclusive)";
        }

        for (unsigned i = 0; i < ratios_.size(); ++i)
            time_step_ratios_[active_time_steps[i]] = ratios_[i];
    }
}
```

Figure 5.8: Example of using the `DoBuild()` function

Moving on to the `DoExecute()` function. This is where the action happens and we apply a survival rate to our categories. The `DoExecute` function is usually called in each year and the time-steps specified in the user input configuration files when running a model.

```

/**
 * Execute the process
 */
void SurvivalConstantRate::DoExecute() {
    LOG_FINEST() << "year: " << model->current_year();

    // get the ratio to apply first
    unsigned time_step = model->managers().time_step()->current_time_step();

    LOG_FINEST() << "Ratios.size() " << time_step_ratios_.size() << " : time_step: " << time_step << " ; ratio: " << time_step_ratios_[time_step]
    Double ratio = time_step_ratios_[time_step];

    StoreForReport("year", model->current_year());

    unsigned i = 0;
    for (auto category : partition_) {
        Double s = s_[category->name_];

        unsigned j = 0;
        LOG_FINEST() << "category " << category->name_ << " ; min_age: " << category->min_age_ << " ; ratio: " << ratio;
        StoreForReport(category->name_ + " ratio", ratio);
        for (Double& data : category->data_) {
            data -= data * (1-exp(-(selectivities_[i]->GetResult(category->min_age_ + j, category->age_length_) * (s * ratio)));
            ++j;
        }
        ++i;
    }
}

```

Figure 5.9: Example of using the DoExecute () function

You will notice another set of logging, which is also encouraged to add all throughout the execution process. The levels of logging are described in Section 4.1

Adding a unit test It is recommended to validate any new functionality using an independent method such as **R** or another stock assessment platform such as Stock Synthesis. For an example using **R** see the file `source\GrowthIncrements\Length\Basix.Test.cpp`. Figure 5.10 has an excerpt which shows the **R** code embedded as a comment within the unit-test. This **R** code will generate expected results that are used by the unit-test in Casal2 to validate functionality. This makes the unit-tests reproducible.

```

CASAL2 > CASAL2 > source > GrowthIncrements > Length > BasicTest.cpp > {} nwa > {} growthincrements
39 | | test the results of our basic growth increment model
40 #' basic growth
41 #' @param length
42 #' @param ga
43 #' @param gb
44 #' @param la
45 #' @param lb
46 #' @return mean length increment
47 #'
48 basic_growth = function(length, ga, gb, la, lb) {
49 | ga + (gb - ga)*(length - la)/(lb - la);
50 | }
51 length_bins = 1:43
52 paste(length_bins, collapse = ", ")
53 la = 20
54 lb = 40
55 ga = 10
56 gb = 1
57 result = vector()
58 for(i in 1:length(length_bins))
59 | result[i] = basic_growth(length_bins[i], ga, gb, la, lb)
60 paste(result, collapse = ", ")
61 | }
62 TEST(GrowthIncrements, Basic) {
63 | shared_ptr<MockModel> model = shared_ptr<MockModel>(new MockModel());
64 | vector<double> lengths
65 | | = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37,
66 | | vector<double> result = {18.55,
67 | | 18.1,
68 | | 17.65,
69 | | 17.2,
70 | | 16.75,
71 | | 16.3,
72 | | 15.85,
73 | | 15.4,
74 | | 14.95,

```

Figure 5.10: Example of R code embedded as a comment within a unit-test

6 Adding reports

This section gives some additional points when adding reports to Casal2.

There are two variables that you must define when describing a report class, these are `model_state_` and `run_mode_`. We will try and explain what values you should give your report, but another avenue is to look at example report classes.

The `model_state_` parameters tells the model when to print the report `State::kExecute` will call the `DoExecute` after each year and `State::kIterationComplete` will print the report once the model has run. So obviously if the report prints annual information or doesn't cache information you will want to use `State::kExecute` e.g. `Partition` else use `State::kIterationComplete`.

Note: if you add a new report that has `model_state_ State::kExecute` you will need to tweak the R-library function `extract.mpd()`. At line 50 of the `of extract.mpd.R` there is the following line of code.

```
1 multi_year_reports = c("partition", "PartitionBiomass", "PartitionMeanWeight")
```

You will need to add the report label type to this vector so that **R** will process the report correctly.

The other command `run_mode_` just tells Casal2 what run-mode should the report be generated in.

7 Merging code changes from a branch to the master

This section describes how to merge changes from a branch to the master repository. See the guidelines in Section 4 for more information on building Casal2.

Note: When merging changes, check that the new code doesn't break any of the existing unit-tests or auto-differentiation code. You will need to build using the commands listed below before merging changes.

1. `doBuild archive` this will build all the autodiff libraries.
2. `doBuild modelrunner`
3. Enter the directory `BuildSystem\Casal2\Casal2 - -unittest` to check all the unit-tests pass.

This section differs from Section 2.4 (where we pulled changes from the master repository into a branch). Here we are merging changes from a branch into the master, which will be incorporated into the master version of Casal2. From this example we can see from Figure 7.1 that our branch is 109 commit ahead and 844 commits behind (underlined in blue) that we want to incorporate into the master repository.

To incorporate these changes into the master you need to click on the “pull request” button. This will prompt a comparison of the changes that you are submitting for inclusion into the master.

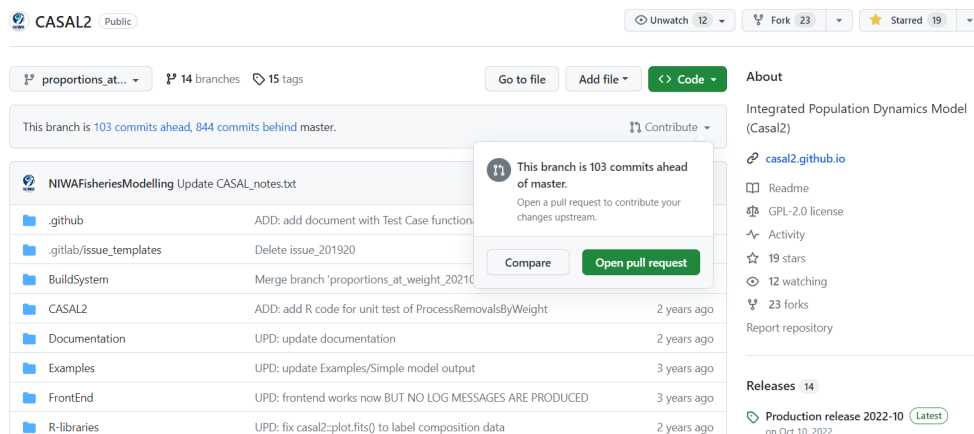


Figure 7.1: Example showing differences between a fork and the master in GitHub

We can see that there are ten files changed from three commits. At the top left corner there is a green button with “create pull request” click that button. This will open a pull request on the master repository as shown in Figure 7.2

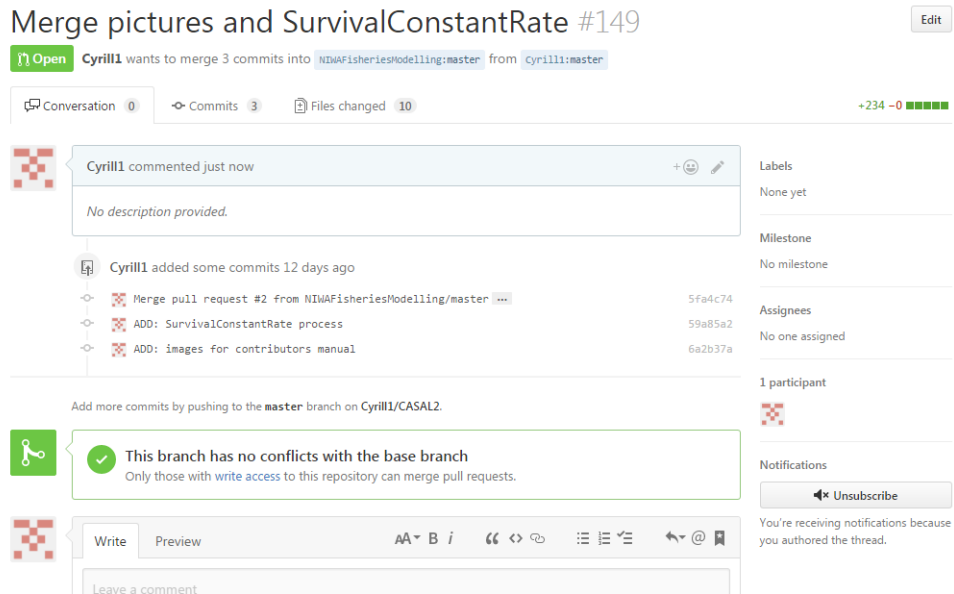


Figure 7.2: Example of a pull request in GitHub

This will notify the maintainers of the master repository that a contributor is requesting a merge of the forked code into the master. Maintainers are able to look through the proposed code changed and will discuss the changes with the contributor if they need clarification. But at this point we are just waiting for the maintainers to accept the changes which then get incorporated into Casal2.

8 Moderating code

When a pull request is received, the Development Team will check the functionality of the code, the syntax, documentation, and any optimisation code required to ensure it runs reasonably. The automated build system test system will report any broken unit-tests or test models.

When making a pull request, please ensure that the submitted changes:

1. Have linted the code to the standard format, and have commented the code to explain the functionality, algorithms, and equations used
2. Use appropriate LOG, WARNING, and ERROR messages to assist any future user and to help with debugging
3. Update the Casal2 User Manual(s), including description of the methods, new functionality and usage, command and subcommand syntax definitions, and adding any index entries as required
4. Include unit tests that validate the changes and additional functionality
5. Include test case models that demonstrate the changes or additional functionality in a simplified use case with a functioning model
6. Pass all of the existing unit tests and the test cases from `doBuild modelrunner`

9 Acknowledgements

We thank the developers of CASAL (Bull et al., 2012) for their ideas that led to the development of Casal2. The Casal2 logo was designed by Ian Doonan and Erika Mackay (NIWA).

Much of the structure of Casal2, the methods and equations, and documentation draw heavily on similar components of the fisheries population modelling application CASAL (Bull et al., 2012) and the spatial population model SPM (Dunn et al., 2021). We thank the authors of CASAL and SPM for their permission to use their work as the basis for parts of Casal2 and allow the use of the definitions, concepts, and documentation.

The Casal2 project is maintained by the Casal2 Development Team. Casal2 was initiated by Alistair Dunn, with Scott Rasmussen as the software architect and lead author of the code. Contributors to the development of Casal2 include Scott Rasmussen, Alistair Dunn, Ian Doonan, Craig Marsh, Teresa A'mar, Kath Large, Sophie Mormede, Samik Datta, Matt Dunn, Jingjing Zhang, Marco Kienze, and Arnaud Grüss.

The development of Casal2 was funded by Earth Sciences NZ (formerly the National Institute of Water & Atmospheric Research Ltd. (NIWA)), with additional funding from Fisheries New Zealand of the Ministry for Primary Industries and Ocean Environmental Ltd.

Casal2 is supported by Fisheries New Zealand, Earth Sciences New Zealand (formerly NIWA), Ocean Environmental Limited, and Scott Rasmussen (Zaita) through the Fisheries New Zealand Casal2 Steering Group.

10 References

- B. Bull, R.I.C.C. Francis, A. Dunn, A. McKenzie, D.J. Gilbert, M.H. Smith, R. Bian, and D. Fu. CASAL C++ Algorithmic Stock Assessment Laboratory): CASAL user manual v2.30-2012/03/21. Technical Report 135, National Institute of Water and Atmospheric Research Ltd (NIWA), 2012.
- A. Dunn, S. Rasmussen, and S. Mormede. Spatial population model user manual, spm v2.03-2021-06-03. Technical report, Ocean Environmental, 2021.